

Aula 10: Algoritmos de Busca e noções de Complexidade

Luís Felipe

UFF

13 de Outubro de 2025

Algoritmos de Busca

- Quando organizamos dados num programa, queremos fazer as seguintes operações: busca, inserção, remoção.
 - ▶ Essas são operações básicas vistas em **Estruturas de Dados**.
 - ▶ Hoje, estamos interessados na **busca** de elementos.
- O problema de busca é caracterizado pela procura de um determinado elemento em um grupo de elementos do mesmo tipo.
- Sem perda de generalidade, o problema será atacado considerando-se que:
 1. Os elementos a serem percorridos são numéricos, distintos e estão armazenados em um vetor;
 2. O número de elementos define o tamanho do vetor;
 3. O elemento procurado pode não estar no vetor (no conjunto de elementos).

Busca de Elemento (Versão Simples)

- Percorre a lista procurando o elemento desejado.
- Retorna o índice da primeira ocorrência ou -1 se não encontrar.

```
1 def buscaElemento(valores, procurado):  
2     local = -1  
3     for indice in range(len(valores)):  
4         if valores[indice] == procurado:  
5             local = indice  
6     return local
```

- Todas as posições do vetor são comparadas com o valor procurado, mesmo quando este é encontrado antes do fim do vetor.
Ex.: [2, 1, 9, 5, 7, 3] e queremos buscar o elemento 1.
- Essa é um algoritmo **ineficiente**, pois podemos fazer menos comparações e terminar antes.

Busca de Elemento (Versão com while)

- Percorre a lista usando while.
- Interrompe a busca assim que encontra o elemento.

```
1 def buscaElemento(valores, procurado):           # primeira solução
2     local = -1
3     indice = 0
4     while indice < len(valores):
5         if valores[indice] != procurado:
6             indice = indice + 1
7         else:
8             local = indice
9             indice = len(valores)                 # força saída do loop
10    return local
```

Busca de Elemento (Segunda Solução com while)

- Percorre a lista usando while.
- Retorna imediatamente o índice quando encontra o elemento.
- Se terminar o laço sem encontrar, retorna -1.

```
1 def buscaElemento(valores, procurado): # segunda solução
2     indice = 0
3     while indice < len(valores):
4         if valores[indice] != procurado:
5             indice = indice + 1
6         else:
7             return indice          # retorna ao encontrar
8     return -1                    # retorna -1 se não encontrar
```

Algoritmo de busca com sentinela

Busca com sentinela:

- Percorre a lista até encontrar o elemento ou chegar ao fim.
- Se encontrar, retorna a posição; caso contrário, retorna 0.

```
1 def busca(lista, x):  
2     i = 0  
3     lista.append(x)           # sentinela  
4     while lista[i] != x:  
5         i += 1  
6     if i != len(lista) - 1:  
7         return i             # encontrado  
8     else:  
9         return 0             # não encontrado
```

- Menos checagens por iteração: em vez de verificar $i < n$ e $lista[i] != x$, você só verifica $lista[i] != x$.

Busca em Lista Ordenada

- Aproveita a ordenação para parar a busca se encontrar elemento maior que o procurado.
- Evita percorrer elementos desnecessários.

```
1 def busca_ordenada(lista, x):  
2     i = 0  
3     lista.append(x)           # sentinela  
4     while lista[i] < x:  
5         i += 1  
6     if i == len(lista) - 1 or lista[i] != x:  
7         return 0             # não encontrado  
8     else:  
9         return i             # encontrado
```

Complexidade

- A noção de **complexidade de algoritmos** se refere ao número de passos (comparações, operações, escritas) feitos pelo algoritmo.

- ▶ Considerando, por exemplo, duas matrizes A e B de dimensões $n \times n$, a soma $C = A + B$ pode ser obtida da seguinte forma:

```
1 for i in range(n):  
2     for j in range(n):  
3         C[i][j] = A[i][j] + B[i][j]
```

- ▶ Assim, independente dos valores que as matrizes A e B possuam, n^2 operações são realizadas.
- ▶ Dizemos então que a complexidade do algoritmo é $O(n^2)$. Definição formal de notação O será vistas em outros cursos, como ED ou APA (http://profs.ic.uff.br/~lfignacio/asa/Aula_2_2023_2.pdf).
- Há, porém, problemas em que um algoritmo executa um número de passos diferente em detrimento a entrada. **Problemas de busca numa lista**, por exemplo.
 - ▶ Note que se o elemento a ser buscado estiver na 1ª posição o algoritmo com sentinela irá realizar 1 passo (será então $O(1)$).
 - ▶ Porém, se o elemento não estiver na lista, o algoritmo vai executar n passos (será então $O(n)$).

Complexidade de Tempo

Sejam:

- A um algoritmo
- $E = \{E_1, E_2, \dots, E_m\}$ o conjunto de todas as entradas possíveis para A
- t_i o número de passos de A dada E_i

Definem-se:

- Complexidade de Pior Caso: $\max_{E_i \in E} t_i$
- Complexidade de Melhor Caso: $\min_{E_i \in E} t_i$
- Complexidade de Caso Médio: $\sum_{1 \leq i \leq m} p_i t_i$, onde p_i denota a probabilidade de ocorrência da entrada E_i .

OBS.: De modo geral, o termo complexidade referir-se-á à complexidade de tempo de pior caso.

Vantagens e desvantagens

- **Pior caso:**
 - ▶ determina o número de passos executados com a entrada mais desfavorável (**muito pessimista**)
 - ▶ quase sempre relevante
 - ▶ a mais utilizada
- **Melhor caso:**
 - ▶ determina o número de passos executados com a entrada mais favorável (**muito otimista**)
 - ▶ em geral, menor relevância
- **Caso médio:**
 - ▶ determina o número de passos de forma proporcional às entradas (**mais justo**)
 - ▶ em geral, relevante
 - ▶ o tratamento é matemático e pode não ser simples
 - ▶ depende das probabilidades de ocorrência das entradas

Complexidades dos algoritmos de busca

- Todos os algoritmos de busca vistos até agora possuem complexidade de melhor caso $O(1)$.
- Todos os algoritmos de busca vistos até agora possuem complexidade de pior caso $O(n)$.

Será que conseguimos um algoritmo que tenha melhor complexidade de pior caso?

Busca Binária

- Requer lista ordenada.
- Divide o intervalo de busca pela metade a cada passo.

```
1 def busca_binaria(lista, x):  
2     inf = 0  
3     sup = len(lista) - 1  
4     while inf <= sup:  
5         meio = (inf + sup) // 2  
6         if lista[meio] == x:  
7             return meio                # encontrado  
8         elif lista[meio] < x:  
9             inf = meio + 1  
10        else:  
11            sup = meio - 1  
12        return -1                    # não encontrado
```

Complexidade do algoritmo de Busca Binária

Análise de complexidade de pior caso:

- No 1o passo, a lista possui n elementos.
- No 2o passo, a lista possui $\frac{n}{2}$ elementos (pois toda uma metade foi removida ao setar o inf ou o sup).
- No 3o passo, a lista possui $\frac{n}{4} = \frac{n}{2^2}$ elementos (mesma justificativa acima).
- No 4o passo, a lista possui $\frac{n}{8} = \frac{n}{2^3}$ elementos.
- Seguindo esse procedimento sucessivamente, no k -ésimo passo, a lista possui $\frac{n}{2^{k-1}}$ elementos.

Note que, no pior caso, o algoritmo terminará quando restar somente 1 elemento. Ou seja, queremos saber qual número k de passos que o algoritmo fará de modo a restar $\frac{n}{2^{k-1}} = 1$ elemento.

Assim, $\frac{n}{2^{k-1}} = 1 \therefore n = 2^{k-1} \therefore \log_2 n = k - 1 \therefore k = O(\log_2 n)$.

$O(\log_2 n)$ contabiliza um número de passos muito menor do que $O(n)$.

Exemplo: $\log_2 1024 = 10$ (veja então como log de um valor é muito menor do que o valor).

Aplicação - Obter a moda

Busca pela moda de uma lista

- A moda de um vetor é o elemento que aparece com mais frequência neste vetor.
- O problema da busca pela moda consiste em encontrar este elemento no vetor.
- Sem perda de generalidade, o problema será atacado considerando-se que:
 - ▶ Os elementos do vetor são numéricos;
 - ▶ Mais de um elemento pode aparecer o mesmo número de vezes e ser moda. Nesse caso, o que aparecer primeiro no vetor será apresentado como resposta.

Exercício: Desenvolva um algoritmo para obter a moda de uma lista, diga qual é esse elemento e em quais posições ele se encontra. Determine também qual a complexidade desse algoritmo.

Algoritmo de obtenção da moda e posições

```
1 def moda_lista(lista):
2     n = len(lista)
3     max_freq = 0
4     moda = None
5     # 1a parte: descobrir qual elemento tem maior frequência
6     for i in range(n):
7         cont = 0
8         for j in range(n):
9             if lista[j] == lista[i]:
10                 cont += 1
11         if cont > max_freq:
12             max_freq = cont
13             moda = lista[i]
14
15     # 2a parte: descobrir as posições da moda
16     posicoes = []
17     for i in range(n):
18         if lista[i] == moda:
19             posicoes.append(i)
20
21     return moda, posicoes
```

```
1 dados = [4, 2, 1, 2, 3, 2, 4, 4, 4]
2 elemento, posicoes = moda_lista(dados)
3 print(f"Moda: {elemento}")           # Moda: 4
4 print(f"Posições: {posicoes}")       # Posições: [0, 6, 7, 8]
```